

Crack the Code of Serverless Design



Grew up in software development



Lots of Programming Languages

BASIC

Microsoft BASIC
GW-Basic
QuickBASIC
Visual Basic
VB.NET

C

C
C++
C#
Java

Niche

JavaScript
Pascal
Python
PHP
COBOL

Software development paradigms

Console Applications

Desktop Applications

Distributed Desktop Applications

Web Applications

Distributed Web Applications

COM

DCMO

.NET

Entity Framework

MVC

Service-Oriented Architecture

Microservices

Cloud-Computing

Serverless Computing

Serverless Computing

Functions-as-a-Service

Event-Driven Architecture

Microservices

Work Experience



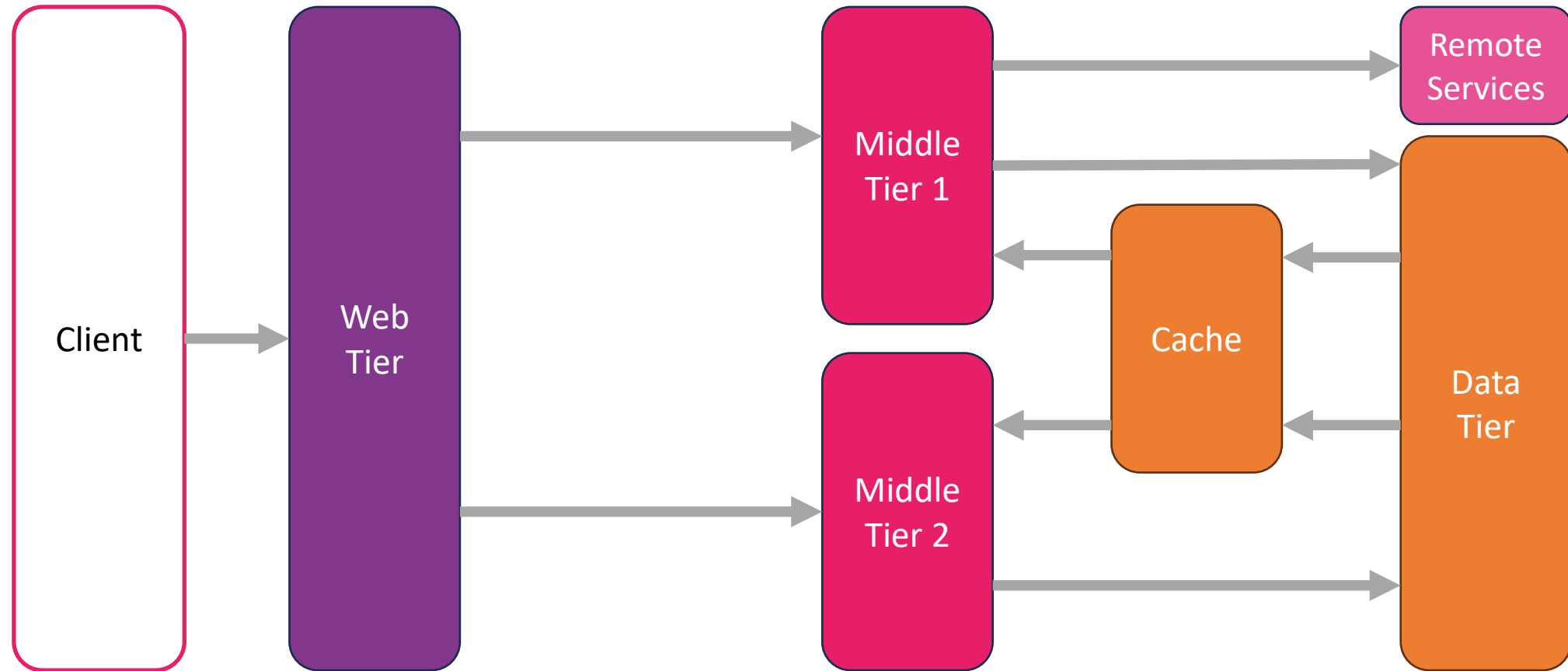
Disclaimer



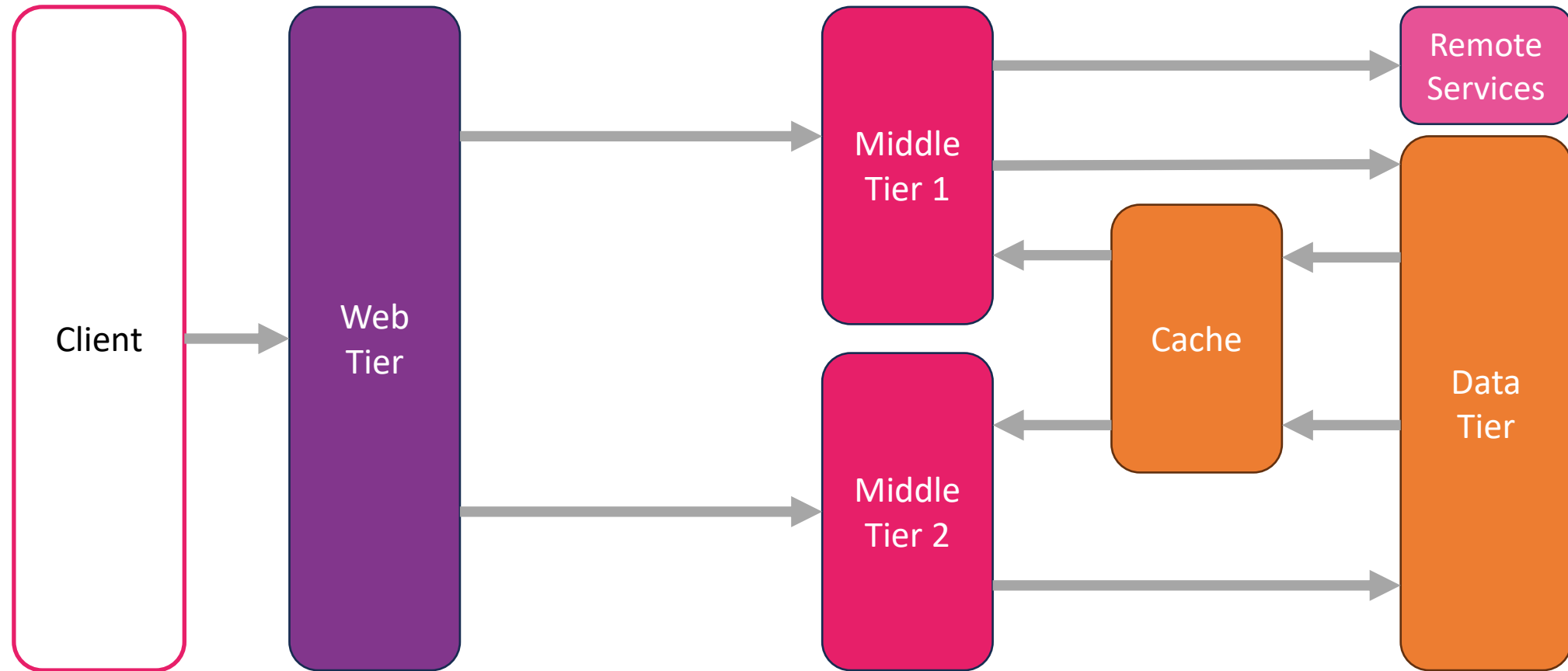
Architecture Styles

Crack the Code to Scalability

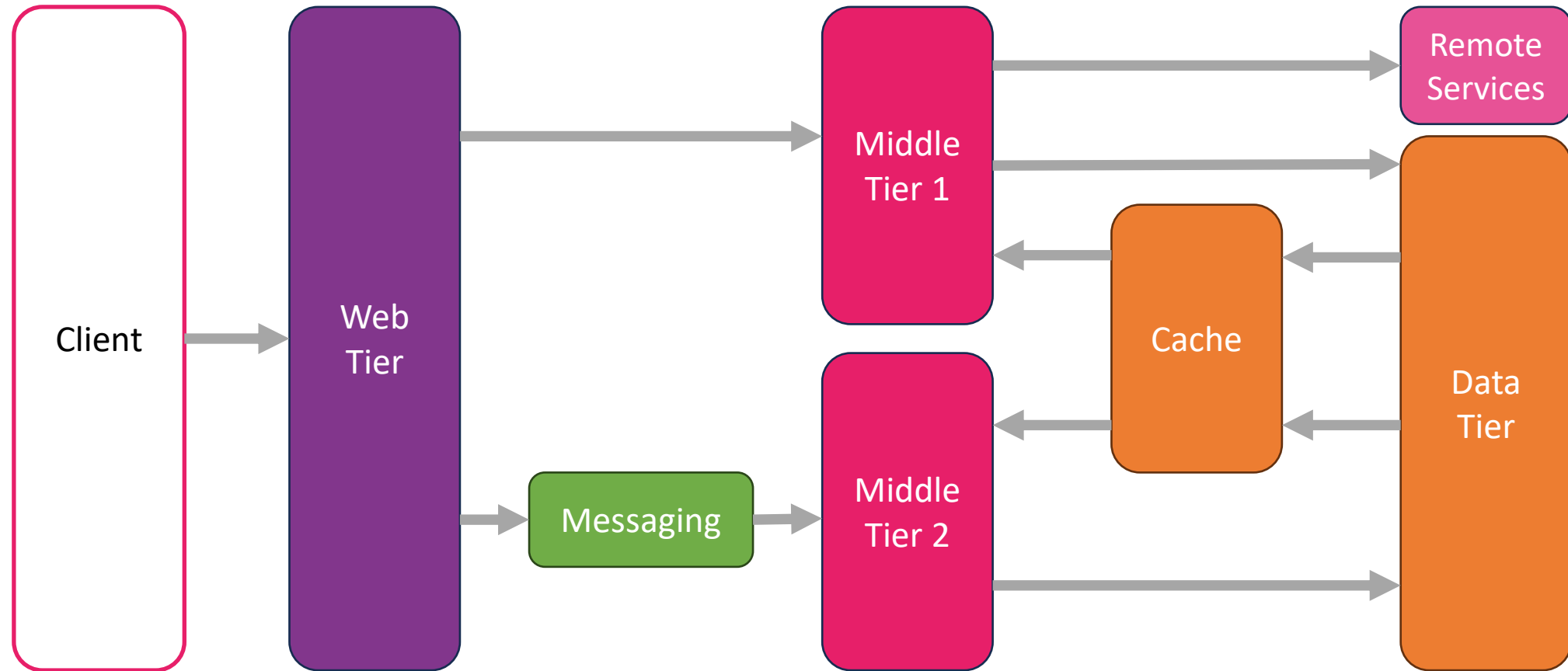
N-Tier



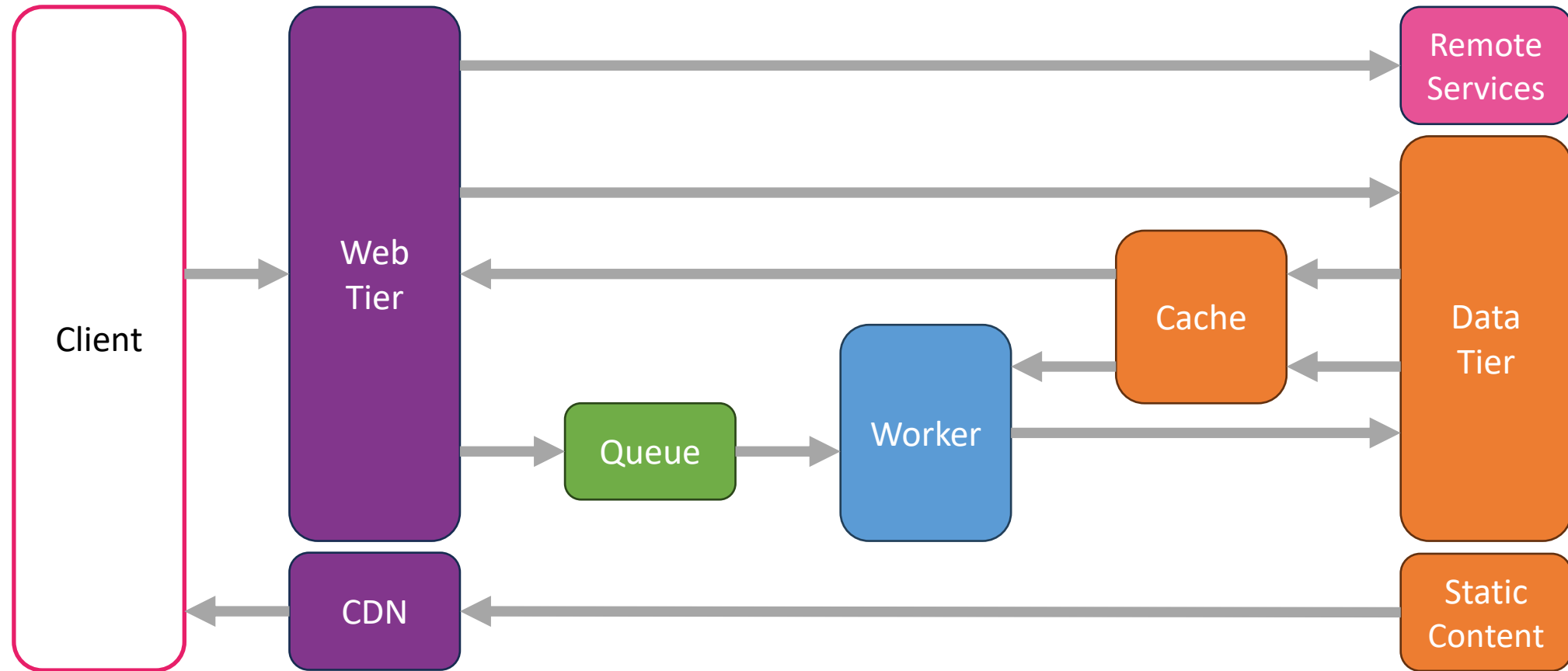
N-Tier



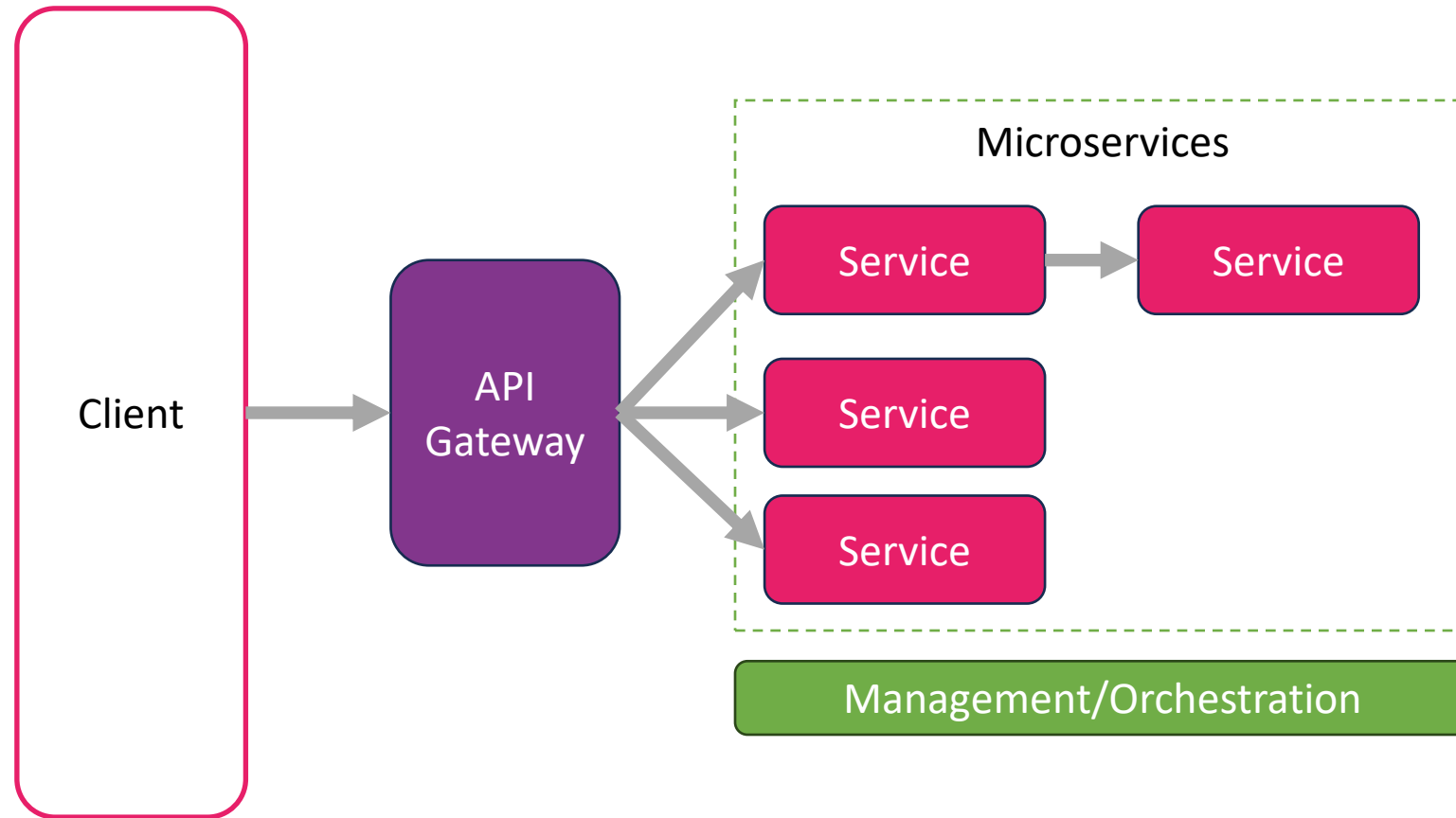
N-Tier with Messaging



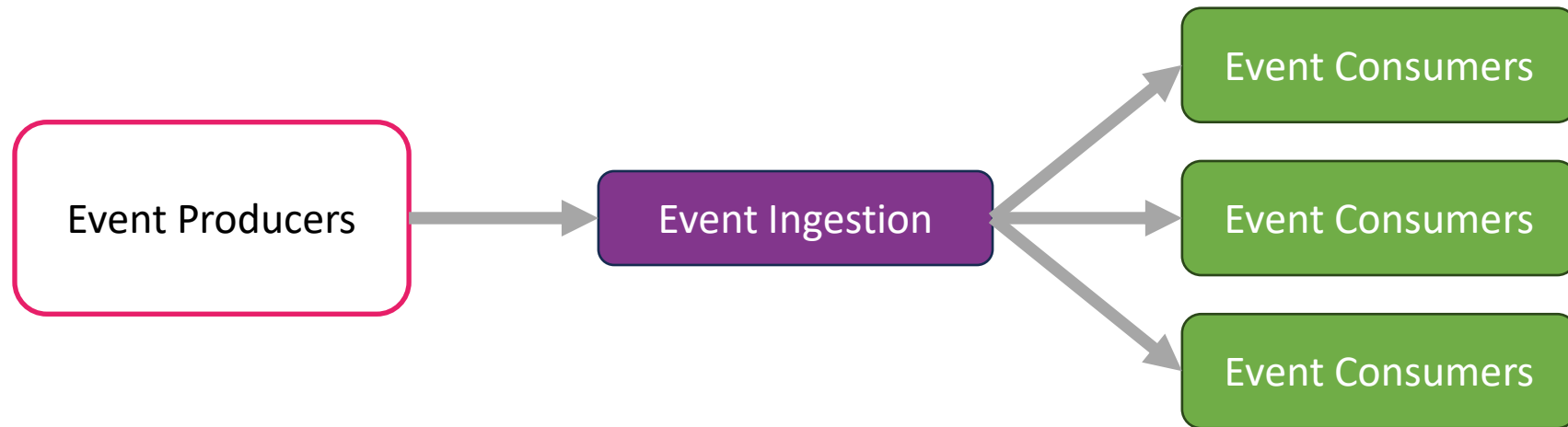
Web-Queue-Worker



Microservices



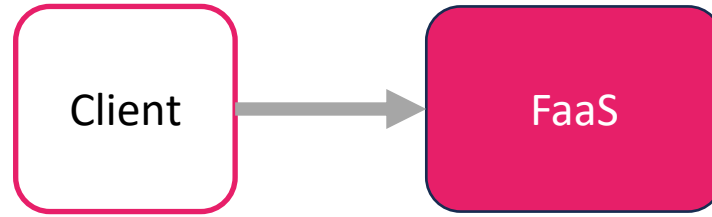
Event-Driven Architecture



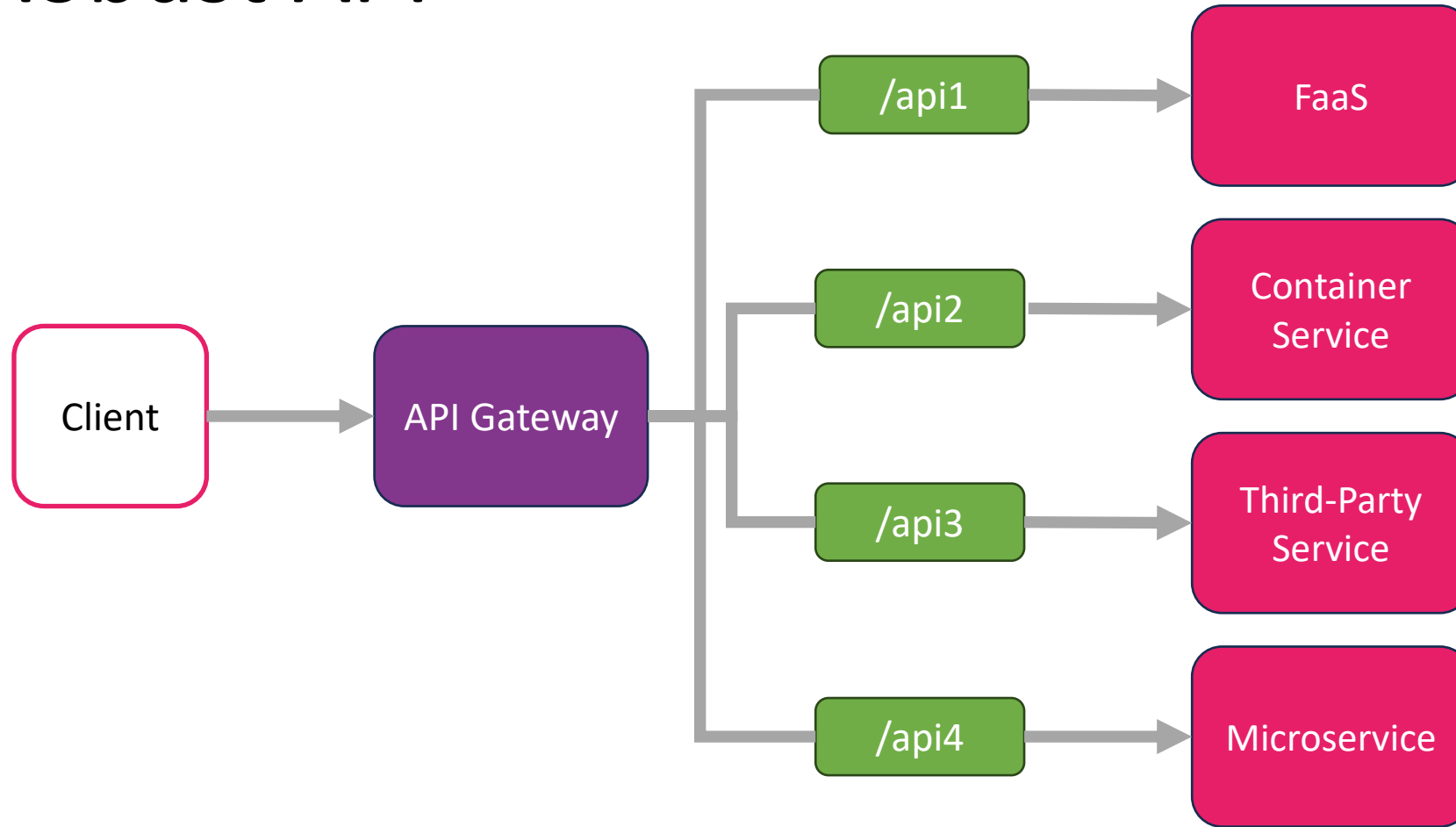
Serverless Design Patterns

Crack the Code to Scalability

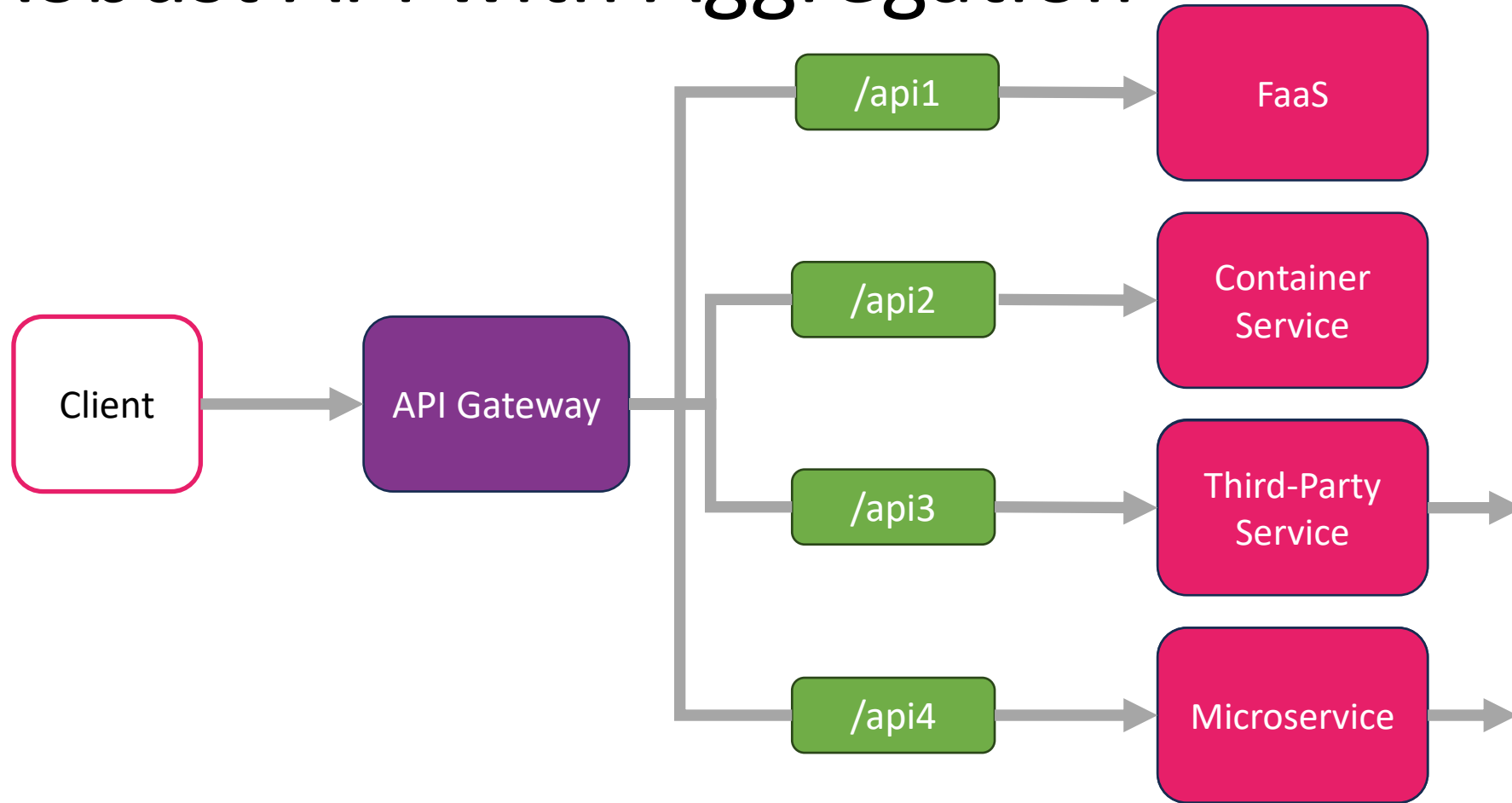
Simple Web Service



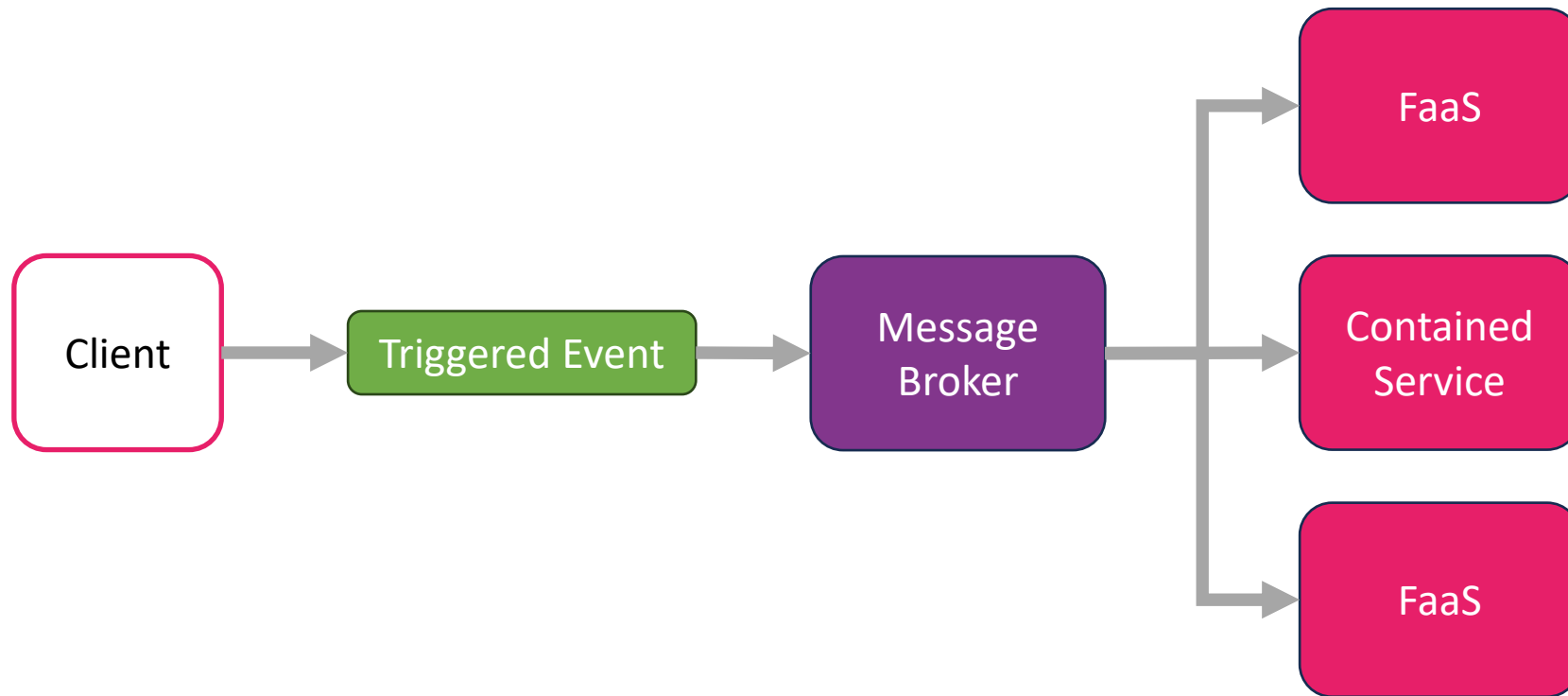
Robust API



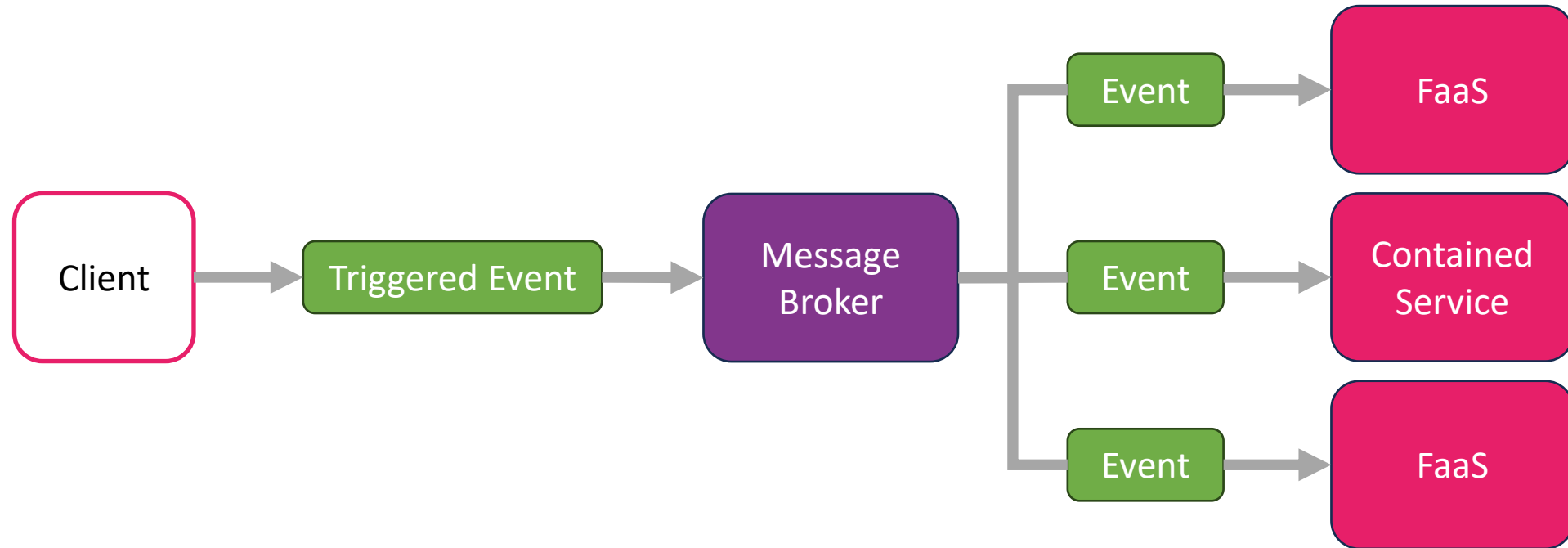
Robust API with Aggregation



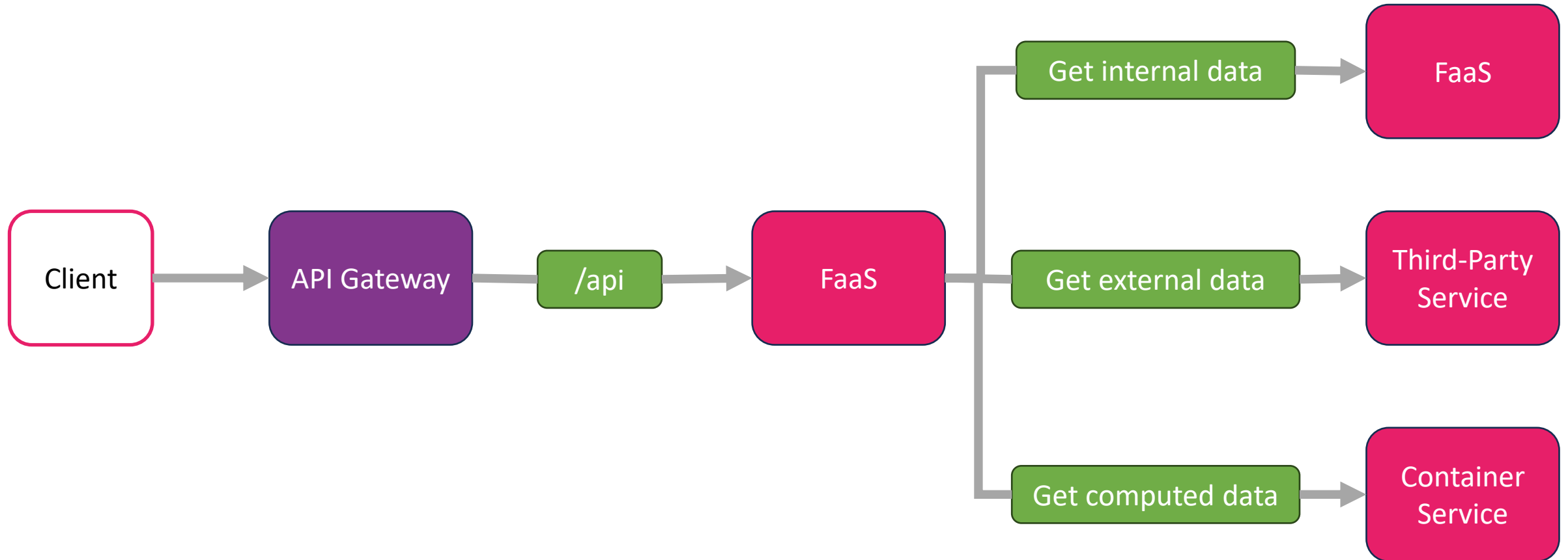
Decoupled Messaging



Publish/Subscribe



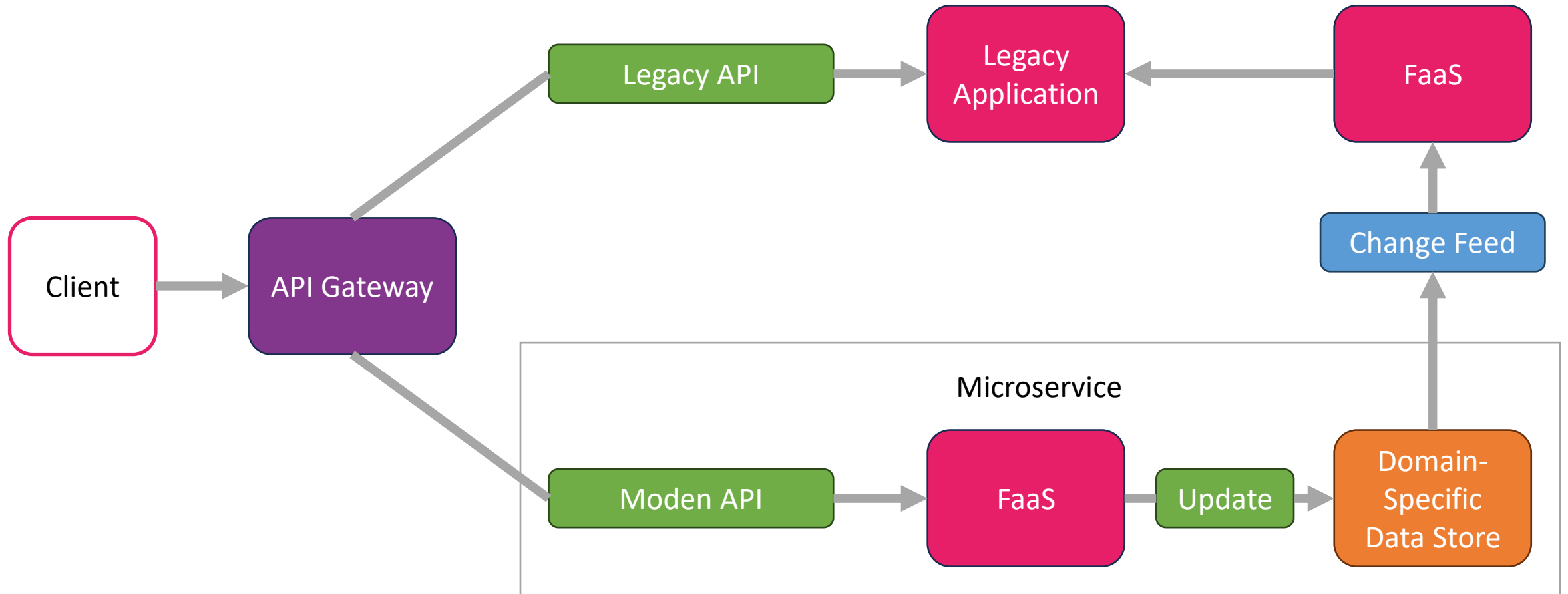
Aggregation



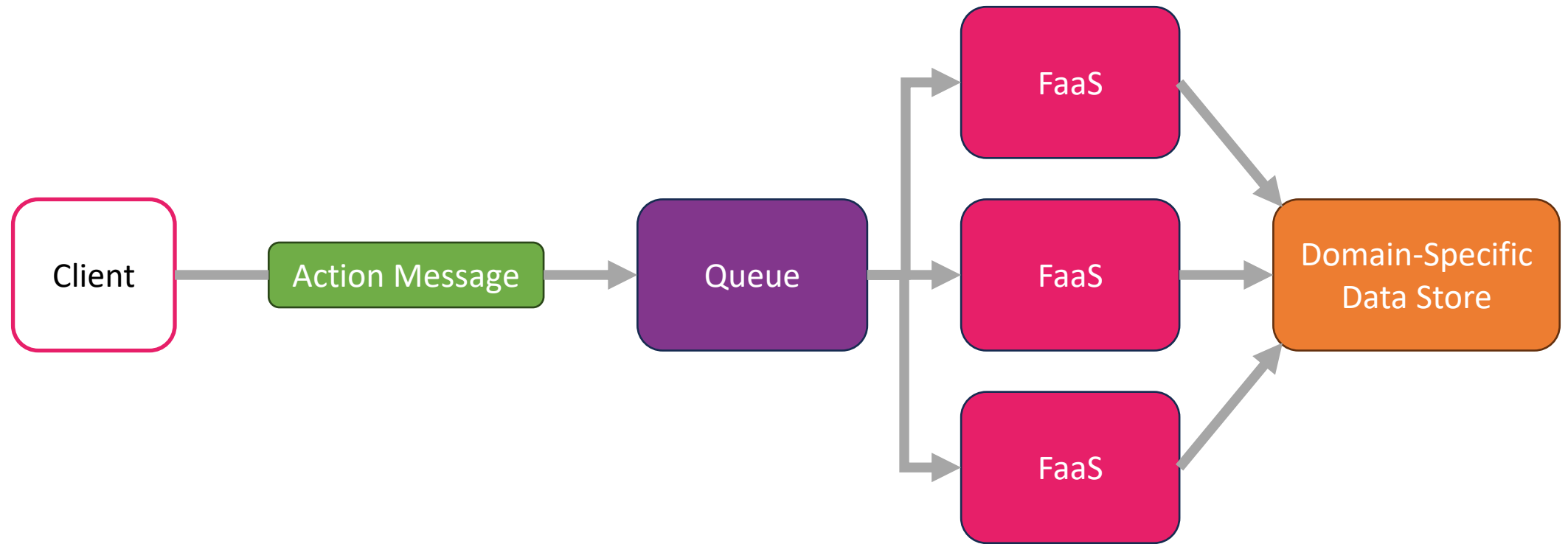
Demo: Aggregation

- A function calling multiple downstream services
- A slow dependency dragging the whole response
- A quick fix (parallel calls, caching, or fallback)

Strangler



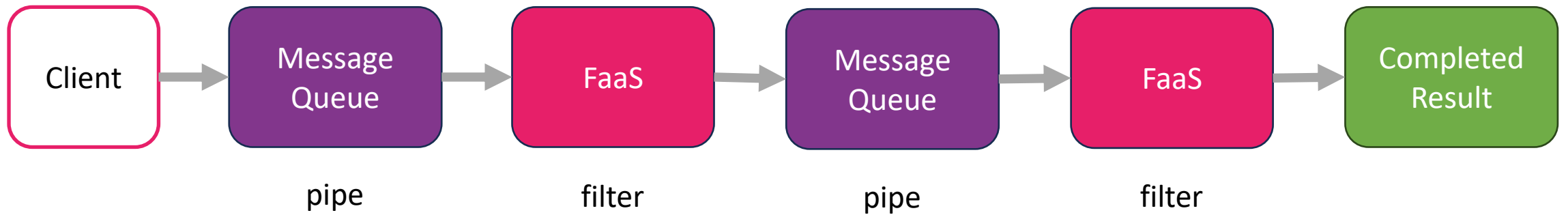
Queue-Based Load Leveling



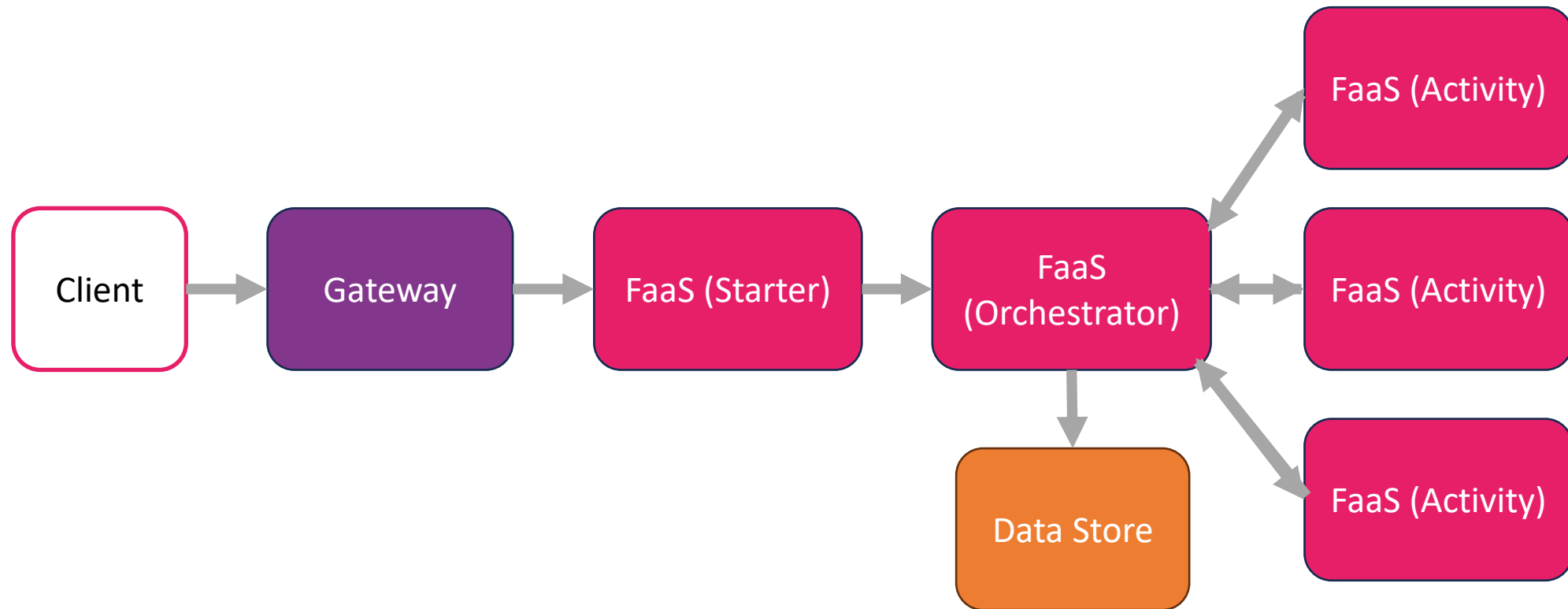
Demo: Queue-Based Load Leveling

- A function that receives a burst of requests
- A queue absorbing the load
- A worker function processing messages at a steady rate
- A poison message scenario

Pipes and Filters



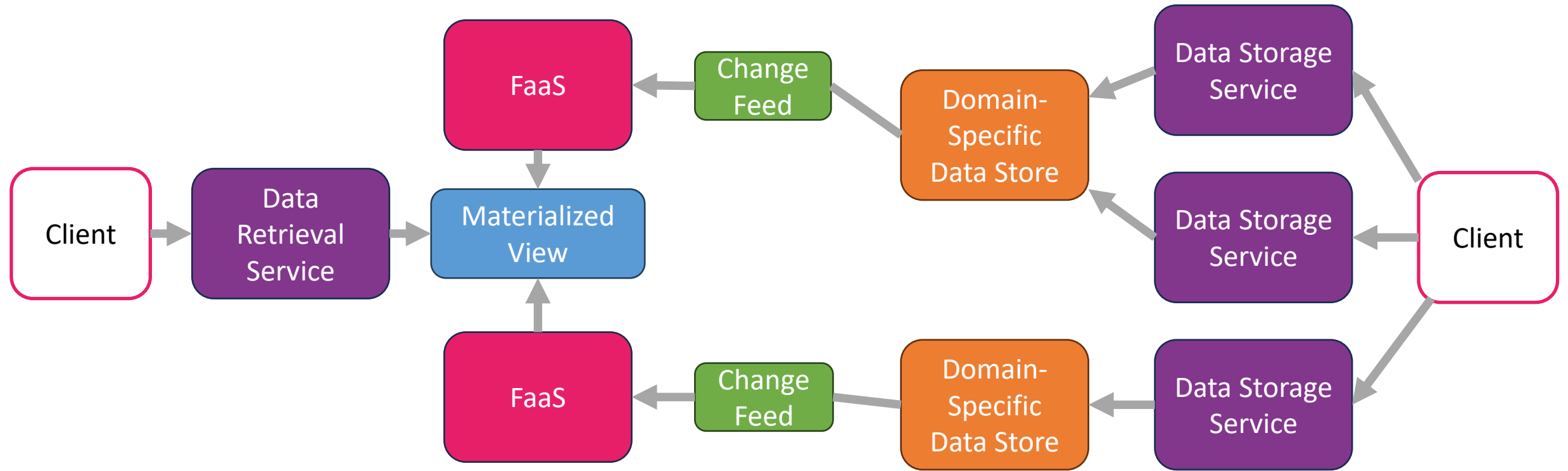
Fan-Out/Fan-In



Demo: Fan-Out/Fan-In

- A starter function kicking off parallel tasks
- A durable orchestrator waiting for all tasks
- A visualization of concurrency
- A quick cost/concurrency discussion

Materialized Views



Pattern Reality Check

Crack the Code to Scalability

Pattern Reality Check

Fan-Out/Fan-In

Aggregation

Pub/Sub

Can explode your costs if you are not careful.

Pattern Reality Check

Fan-Out/Fan-In

Aggregation

Pub/Sub

Your API is only as
fast as your slowest
dependency

Pattern Reality Check

Fan-Out/Fan-In

Aggregation

Pub/Sub

Pub/Sub is NOT a workflow engine.

Serverless Best Practices

Crack the Code to Scalability

Serverless Best Practices

Stay Focused

Serverless Best Practices

Stay Focused

Handle Errors

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Log and Trace

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Log and Trace

Versioning and
Deployment

Serverless Best Practices

Stay Focused

Handle Errors

Observe What's
Going On

Test, Test, and
Test Again

Optimize Costs

Engage Shields

Log and Trace

Versioning and
Deployment

Short Interactive Exercise

Crack the Code to Scalability

You need to process 50,000 incoming images,
enrich them with metadata, and store them.
Which pattern would you choose?

Summary and Key Takeaways

Crack the Code to Scalability

Q&A



Crack the Code to Scalability

Stay Connected

✉ chadgreen@chadgreen.com

💬 TaleLearnCode

🌐 ChadGreen.com

🐦 ChadGreen

🌐 ChadwickEGreen

